

DEPARTMENT OF TRAINING AND PLACEMENT
Engineering Programme · School of Computer Science and Engineering

TCS National Qualifier Test Training Programme

A Twenty-Day Preparation Schedule for Engineering Students

ACADEMIC SESSION 2026 – 2027

DURATION: Twenty Training Days

TOTAL HOURS: 120 Hours (6 Hours per Day)

SECTIONS COVERED: Aptitude & Reasoning, Data Structures & Algorithms

TRAINING PARTNER

PINAKI IT CONSULTANT PVT. LTD.

ISO 9001:2015 | IAF Accredited | MSME Registered
www.pinakiithub.com | +91 9540855058

Prepared by the Department of Training and Placement · For Internal Academic Use

Preface

Foreword

Campus recruitment through the TCS National Qualifier Test has become one of the more demanding stages in a student's academic journey. The test evaluates quantitative aptitude, verbal ability, logical reasoning, and programming proficiency within a fixed time window, and performance in each section is assessed independently. Many students who are otherwise strong in their core engineering subjects find this format unfamiliar, since it does not always follow the pace or structure of regular coursework.

The Department of Training and Placement, in partnership with PINAKI IT CONSULTANT PVT. LTD., has designed this twenty-day training programme to address that gap. The intention is not to introduce new academic content, but to help students consolidate what they already know and apply it under examination conditions. Each day is divided between aptitude and coding practice, so that students build both areas together rather than treating them as separate preparations undertaken close to the recruitment drive.

A programme of this kind is most useful when it is followed with regularity. A single day of intensive preparation, however focused, does not build the recall speed or problem-solving instinct that a timed assessment requires. For this reason, the schedule has been sequenced so that each day builds on the material covered previously, moving from foundational topics toward more advanced problem types, and concluding with revision and full-length mock tests in the final stretch of the programme.

This handbook is meant to serve as a reference for both students and faculty coordinating the sessions. It sets out what will be covered on each day, what is expected of students in terms of practice, and how progress will be assessed over the course of the programme. Students are encouraged to treat the daily assignments with the same seriousness as the classroom sessions, since consistent practice outside class hours has a direct bearing on performance in the actual test.

It is hoped that this programme gives students the preparation and confidence required to perform well in the TCS NQT, and in the similar assessments many will encounter through the placement season.

Department of Training and Placement

In association with PINAKI IT CONSULTANT PVT. LTD.

Section 1 — Program Overview

Purpose

This programme has been designed to prepare pre-final and final-year engineering students for the TCS National Qualifier Test. It addresses the two components evaluated in the test — quantitative aptitude with reasoning, and programming with data structures — and is structured to move students from foundational concepts to advanced problem types over a fixed training period.

Target Audience

The programme is intended for students across engineering disciplines who are eligible for placement through TCS or similar recruitment drives. It assumes a working knowledge of basic mathematics and at least one programming language, both of which are typically covered in the earlier years of an engineering curriculum.

Training Duration

The programme runs over twenty consecutive training days. Each day comprises six hours of instruction, divided equally between aptitude and coding, for a total of 120 hours of classroom engagement.

Delivery Method

Sessions are conducted in a classroom setting and combine instructor-led explanation with supervised practice. Each day includes a discussion of the day's topic, a demonstration of problem-solving methods, and time for students to attempt problems independently while faculty remain available for guidance. Coding sessions are conducted on a computer with access to an IDE or online judge. Training delivery is coordinated jointly with PINAKI IT CONSULTANT PVT. LTD..

Expected Outcomes

By the end of the programme, students are expected to solve aptitude questions within the time limits typically set by the test, and to write and debug code for standard data structure and algorithm problems without external reference. Further detail on outcomes is provided in Section 7 of this handbook.

Section 2 — Program Highlights

The following points summarise the structure of the programme at a glance.

Twenty Training Days: Consecutive daily sessions, sequenced from foundational to advanced material.

120 Contact Hours: Six hours of instruction per day, split evenly between the two tracks.

Two Parallel Tracks: Aptitude and reasoning in the morning session; coding and data structures in the afternoon.

Forty-Eight Topics: Twenty-four aptitude topics and twenty-four DSA topics, covered without repetition.

Three Mock Assessments: Full-length tests in the final three days, modelled on the TCS NQT pattern.

Daily Assignments: Practice problems set after every session and reviewed at the start of the next.

Section 3 — Training Methodology

Each training day follows the same instructional pattern, regardless of the topic being covered. This consistency allows students to anticipate the structure of the session and focus their attention on the material itself rather than on the format of the class. The six components of a typical day are described below.

Component	Approx. Duration	Description
Concept Learning	30–40 minutes	Introduction of the day's topic, including definitions, formulae, or the underlying idea behind a data structure or algorithm.
Instructor Demonstration	30–40 minutes	Worked examples solved on the board or in an IDE, showing the reasoning behind each step.
Guided Practice	40–50 minutes	Students attempt similar problems while faculty circulate and address individual queries.
Coding Session	60–90 minutes	Hands-on implementation of the day's DSA topic on a computer, using an IDE or an online judge.
Assignments	Take-home	A set of problems assigned for independent practice, to be reviewed at the start of the following session.
Doubt Resolution	15–20 minutes	Time set aside at the end of the day for questions arising from the session or from the previous day's assignment.

The proportion of time given to each component varies slightly depending on whether the day is weighted toward aptitude or toward coding, but the overall sequence — explanation, demonstration, supervised practice, independent work, and review — remains constant throughout the programme.

Section 4 — 20-Day Training Schedule

Days 1–8 · Foundational Topics · 3 Hours Aptitude + 3 Hours DSA/Coding per day

Day	Aptitude Topics	DSA / Coding Topics	Learning Outcomes	Practice (Aptitude + Coding)	Homework
1	Number System (divisibility, LCM/HCF, remainders, factors)	Programming Basics (I/O, variables, operators, control flow) + Complexity Analysis (Big-O, time/space)	Build numeric problem-solving foundation; understand code efficiency evaluation	20 Number System MCQs; 5 basic I/O programs; identify complexity of 5 code snippets	15 Number System problems; write complexity of 3 given algorithms
2	Percentages (basics, successive %, applications)	Arrays – Fundamentals (declaration, traversal, insertion/deletion)	Convert real-world quantities to percentage change; manipulate 1D arrays confidently	20 Percentage problems; solve 5 array traversal problems on judge	15 Percentage word problems; 5 array basic coding problems
3	Ratio & Proportion (direct/inverse, partnership)	Arrays – Advanced (rotation, subarrays) + Two Pointers	Apply ratio logic to partnership/mixture-type problems; solve array problems in optimal time using two pointers	20 Ratio/Proportion MCQs; 5 two-pointer coding problems (pair sum, reverse array)	15 Ratio problems; 5 two-pointer LeetCode-style problems
4	Average (basic, weighted, age-related)	Strings – Fundamentals (traversal, palindrome, reversal, anagram check)	Solve average-based puzzles quickly; perform core string operations	20 Average problems; 5 string manipulation problems	15 Average problems; 5 string coding problems
5	Profit & Loss (basic, discount, successive discount)	Searching (Linear Search, Binary Search + variants)	Compute profit/loss/discount scenarios confidently; implement and apply binary search on sorted data	20 Profit & Loss MCQs; 5 searching problems (first/last occurrence, rotated array search)	15 Profit/Loss problems; 5 binary search coding problems
6	Simple Interest & Compound Interest	Sorting (Bubble, Selection, Insertion, Merge, Quick)	Differentiate SI vs CI applications; understand and implement major sorting algorithms with complexity trade-offs	20 SI/CI problems; implement 3 sorting algorithms from scratch	15 SI/CI problems; 2 sorting implementation + 3 sorting-based coding problems
7	Time & Work (individual, combined, efficiency-based)	Recursion (basics, factorial, Fibonacci, recursion tree)	Solve work-efficiency problems using LCM method; write and trace recursive functions	20 Time & Work problems; 5 recursion problems	15 Time & Work problems; 5 recursion coding problems
8	Time, Speed & Distance (relative speed, trains, boats & streams)	Hashing (HashMap/HashSet usage, frequency counting, collision handling)	Solve motion-based problems including relative speed cases; use hashing to optimise $O(n^2)$ problems to $O(n)$	20 TSD problems; 5 hashing-based coding problems	15 TSD problems; 5 hashmap/frequency-count problems

Days 9–17 · Intermediate to Advanced Topics · 3 Hours Aptitude + 3 Hours DSA/Coding per day

Day	Aptitude Topics	DSA / Coding Topics	Learning Outcomes	Practice (Aptitude + Coding)	Homework
9	Pipes & Cisterns + Mixtures & Alligations	Sliding Window + Prefix Sum	Combine work-rate logic with mixture ratios; solve subarray/substring problems using window and prefix-sum techniques	20 combined problems (Pipes/Mixtures); 5 sliding window + 5 prefix sum problems	15 Pipes/Mixtures problems; 5 sliding window/prefix sum coding problems
10	Probability (basic events, cards, dice)	Linked Lists (singly, doubly, insertion/deletion, reversal)	Compute probability of simple/compound events; implement and manipulate linked list structures	20 Probability problems; 5 linked list coding problems (reverse, detect loop)	15 Probability problems; 5 linked list problems
11	Permutation & Combination	Stacks & Queues (array/linked list based, applications)	Apply P&C formulas to arrangement/selection problems; use stacks/queues to solve balanced-parentheses, next-greater-element type problems	20 P&C problems; 5 stack/queue coding problems	15 P&C problems; 5 stack/queue application problems
12	Data Interpretation (tables, bar/pie/line graphs)	Trees – Fundamentals (binary tree, traversals: inorder/preorder/postorder/level-order)	Extract and analyse data from graphical/tabular formats; implement tree traversals recursively and iteratively	3 full DI sets (15 questions); 5 tree traversal coding problems	2 DI sets; 5 tree-based coding problems
13	Logical & Analytical Reasoning (series, analogy, classification)	Binary Search Tree (insertion, deletion, search, validation)	Identify logical patterns and sequences; construct and validate BSTs efficiently	20 reasoning problems; 5 BST coding problems	15 reasoning problems; 5 BST problems
14	Coding-Decoding + Blood Relations	Heap (min-heap, max-heap, heapify, priority queue applications)	Decode letter/number patterns; solve family-tree relationship puzzles; implement heap-based problems (Kth largest, etc.)	20 combined problems; 5 heap coding problems	15 Coding-Decoding/Blood Relations problems; 5 heap problems
15	Direction Sense + Seating Arrangement (linear/circular)	Graphs – Fundamentals (representation, BFS, DFS)	Solve directional and arrangement-based puzzles systematically; implement graph traversal algorithms	20 combined problems; 5 BFS/DFS coding problems	15 arrangement/direction problems; 5 graph traversal problems
16	Puzzles (grid-based, box puzzles) + Syllogism	Greedy Algorithms + Dynamic Programming (Basics: Fibonacci DP, Knapsack intro, coin change)	Approach complex puzzles methodically; differentiate greedy vs DP approach; solve basic optimisation problems	15 puzzle/syllogism problems; 5 greedy + 5 basic DP problems	10 puzzle problems; 5 greedy/DP coding problems
17	Verbal Ability + Reading Comprehension + Grammar + Vocabulary	Backtracking + Bit Manipulation + Previous TCS Coding Questions	Improve English proficiency (grammar accuracy, vocabulary, comprehension speed); solve constraint-satisfaction problems using backtracking; apply bitwise tricks; solve authentic TCS-pattern coding questions	2 RC passages; 20 grammar/vocab MCQs; 5 backtracking + 5 bit manipulation problems; 5 previous TCS coding questions	1 RC set; 15 grammar/vocab problems; 5 backtracking/bit manipulation problems

Days 18–20 · Revision, Mock Assessments and Interview Preparation

Day	Aptitude Topics	DSA / Coding Topics	Learning Outcomes	Practice (Aptitude + Coding)	Homework
18	Full Revision – Arithmetic (Number System to Probability) + Full Revision – Reasoning & Verbal	Full Revision – Arrays, Strings, Searching, Sorting, Recursion, Hashing	Consolidate all quantitative, reasoning, and verbal concepts; revisit core DSA implementation patterns	1 full-length Aptitude mock (Quant + Reasoning + Verbal); 10 mixed coding problems from Days 1–9	Revise error log from Days 1–9; redo 10 previously incorrect problems
19	Mixed Practice & Full-Length Mock Test (TCS NQT pattern: Quant + Reasoning + Verbal)	Full Revision – Linked Lists, Stacks/Queues, Trees, BST, Heap, Graphs, Greedy, DP, Backtracking, Bit Manipulation	Simulate real exam conditions with timed sectional and full mock; reinforce advanced DSA concepts through mixed problem sets	1 full-length aptitude mock with analysis; 10 mixed advanced coding problems	Analyse mock test report; identify and document top 5 weak areas
20	Grand Mock Test + Interview-Oriented Problem Solving + Doubt Clearing Session	Coding Contest (Timed) + Interview-Oriented Problem Solving + Doubt Clearing Session	Build exam-day confidence and time management; simulate technical interview problem-solving under pressure; resolve outstanding conceptual doubts	1 complete TCS NQT-pattern mock (Aptitude + Coding, timed); 1 timed coding contest (5 problems, TCS interview-style); open Q&A / doubt-clearing	Prepare personal formula sheet and coding pattern reference; review complete error log from all 20 days

Shaded rows (Days 18–20) mark the revision and assessment phase of the programme.

Section 5 — Assessment Strategy

Assessment through the programme is continuous rather than confined to the closing days. Daily practice is reviewed as it is completed, so that gaps in understanding are identified while the material is still fresh. Sectional and full-length mock tests are placed at fixed points in the schedule to give students an accurate sense of their readiness before the actual test.

Assessment	Frequency	Purpose
Daily practice sets (aptitude MCQs and coding problems)	Every day, Days 1–17	Reinforce the day's topic immediately after instruction
Topic-wise quizzes	Every 4–5 days	Check retention across a cluster of related topics
Homework review and error log	Daily	Track recurring mistakes and close individual gaps
Sectional mock tests (quant, reasoning, verbal, coding)	Days 9, 13, 17	Build sectional speed under exam-like time limits
Full-length mock tests (TCS NQT pattern)	Days 18, 19, 20	Simulate the complete test under timed conditions
Timed coding contest	Day 20	Assess coding speed and accuracy under pressure
Interview-oriented problem solving	Day 20	Prepare students for technical interview questions
Doubt-clearing and feedback session	Day 20	Resolve outstanding queries and record individual feedback

Performance is tracked through accuracy, average time per question, and results against the sectional cut-offs typically applied in the TCS NQT. This information is shared with each student individually so that the remaining days of the programme can be used to address specific weak areas.

Section 6 — Expected Learning Outcomes

By the end of the programme, a student should be able to:

1. Solve aptitude questions across all topics from Number System through Data Interpretation within the time limits set by the actual test.
2. Apply logical, analytical, and verbal reasoning techniques to puzzles, arrangements, syllogisms, and comprehension passages.
3. Read and respond to English-language questions on grammar, vocabulary, and comprehension with reasonable accuracy and speed.
4. Write working code for arrays, strings, linked lists, stacks, queues, trees, graphs, and heaps, and explain the time and space complexity of each solution.
5. Apply searching, sorting, recursion, greedy methods, dynamic programming, and backtracking to both familiar and unfamiliar problems.
6. Use two pointers, sliding window, prefix sum, hashing, and bit manipulation to improve the efficiency of a brute-force solution.
7. Complete a full-length, sectional-timed mock test with fewer errors and less time spent per question than at the start of the programme.
8. Discuss a coding problem out loud, explaining the approach before and while writing the solution, as expected in a technical interview.
9. Identify personal weak areas from the error log maintained through the programme and continue to address them independently after the programme ends.

Section 7 — Student Deliverables

Students leave the programme with a set of materials they have prepared themselves through the twenty days, in addition to the record maintained by the department.

Deliverable	Description
Daily worksheets	Completed aptitude and coding practice sets for each of the twenty training days.
Personal error log	A running record of mistakes made across topics, maintained by the student and reviewed daily.
Mock test scorecards	Individual results and section-wise analysis for each of the three full-length mock tests.
Formula and pattern reference	A personal sheet of aptitude formulae and common coding patterns, compiled by the student over the course of the programme.
Attendance and performance record	Maintained by the department for reference during placement coordination.
Certificate of participation	Issued jointly by the Department of Training and Placement and PINAKI IT CONSULTANT PVT. LTD. on satisfactory completion of the programme.

Closing — Conclusion

This handbook sets out the structure, content, and assessment plan for the twenty-day TCS NQT training programme. The schedule has been built to cover the full range of topics tested in the examination, without repetition, and in an order that allows each day to build on the one before it.

The value of the programme depends, in large part, on consistent participation. Students who attend each session, complete the assigned practice, and use the mock tests to identify and correct weak areas are better placed to perform well in the actual assessment. Faculty coordinating the sessions are encouraged to track individual progress through the error logs and mock test scorecards described in this handbook, and to raise concerns about a student's readiness well before the recruitment drive.

Questions regarding scheduling, content, or individual student progress may be directed to the Department of Training and Placement or to the training team at PINAKI IT CONSULTANT PVT. LTD..

Department of Training and Placement

In association with PINAKI IT CONSULTANT PVT. LTD.

PINAKI IT CONSULTANT PVT. LTD.

ISO 9001:2015 | IAF Accredited | MSME Registered

www.pinakiithub.com | +91 9540855058